A SYNOPSIS REPORT ON

MERIDIAN — AI MULTI-AGENT MARKET INTELLIGENCE PLATFORM

Submitted in partial fulfilment of the Requirements for the award of

DEGREE OF BACHELOR OF COMPUTER APPLICATION

SUBMITTED BY

Name: Mridul Sharma

Class: BCA - 5

University Roll No: 2430087

DEPARTMENT OF COMPUTER APPLICATIONS

GLOBAL GROUP OF INSTITUTES, AMRITSAR

AUGUST – DECEMBER, 2026

TABLE OF CONTENTS

Chapter No.	CONTENTS	PAGE NO.
1.	Introduction to Project	1
2.	Objectives and Scope of Project	3
3.	System Architecture & Feasibility Study	4
4.	Introduction to HTML5, CSS3, JavaScript & Python	6
5.	Hardware and Software Requirements	7
6.	Conclusion & Expected Outcomes	8
7.	References	8

1. INTRODUCTION TO PROJECT

Market intelligence — the disciplined collection and analysis of information about markets, competitors, customers, and macroeconomic forces — has become a decisive input to business strategy. Consulting-grade research reports that once required teams of analysts and weeks of effort are now expected within hours, and decision makers increasingly ask follow-up questions of the data itself. At the same time, general-purpose Artificial Intelligence (AI) chatbots frequently invent figures, fabricate citations, and present guesses as verified facts, which makes them unsafe for board-level decisions.

Meridian — AI Multi-Agent Market Intelligence Platform is a full-featured, multi-agent web application (Software-as-a-Service architecture) engineered using Python (FastAPI), HTML5, CSS3, Vanilla JavaScript (ES6+), and SQLite / PostgreSQL that solves this problem end-to-end. A user simply enters a research query such as “India's economy growth compared to China”, selects a Region, Period, and Language, and clicks Generate Intelligence. Within moments, seven autonomous AI agents — Researcher, Analyst, Writer, Forecast Agent, Fact-Checker, Chat Agent, and Translator — collaborate to research live sources, run SWOT / PESTLE / Porter's Five Forces analyses, forecast twelve months of scenarios, fact-check every claim, and compose a board-ready report with real citations, interactive charts, and one-click PDF, PowerPoint, and Excel exports.

A defining architectural innovation of Meridian is its Verified-Answer Policy combined with a Bring-Your-Own-Key (BYOK) multi-provider engine. Users connect free-tier API keys from sixteen supported providers — OpenRouter (free models), Groq, Google Gemini, Cerebras, Mistral, Cohere, OpenAI, Anthropic, and others — directly inside the Settings interface. Keys are encrypted at rest with Fernet (AES-128-CBC + HMAC) cryptography, and no answer is ever manufactured from templates: if no API key or no live evidence is available, Meridian refuses with a clear, structured error instead of publishing fabricated research.

Project Development Methodology:

The project was conceptualized and engineered systematically through the following phases:

- **Requirement Gathering & Problem Formulation:** Identifying the cost, delay, and hallucination problems of manual market research and generic AI chatbots.
- **System & UI/UX Architecture Design:** Designing a multi-view web application (Dashboard, Report View, Report History, Compare Mode, API Keys, and User Guide) styled in a three-colour Obsidian, Champagne Gold, and Pearl palette.

• **Multi-Agent Engine Implementation:** Developing the seven-agent orchestration pipeline (Researcher, Analyst, Writer, Forecast, Fact-Checker, Chat, Translator) in Python with a resilient multi-provider LLM router featuring free-model auto-selection, fallback chains, and rate-limit survival.

• **Frontend Interactivity & Validation Coding:** Implementing live Server-Sent Events (SSE) mission-control telemetry, interactive charts, encrypted key management, PDF/PPTX/XLSX export generation, and toast-based error handling in Vanilla JavaScript.

• **Testing, Debugging & Academic Documentation:** Executing 123 automated test cases across the agent pipeline, provider router, and HTTP API, and preparing comprehensive project documentation.

Multi-View Structure of Meridian:

Route	Page Name	Primary Role
#/dashboard	Dashboard	Query composer, live KPI cards, market charts, and agent mission control.
#/report/{id}	Report View	Full cited report with charts, SWOT/PESTLE/Porter tables, forecast, and export
#/history	Report History	Searchable archive of generated reports with reliability scores.
#/compare	Compare Mode	Side-by-side comparison of two companies or past reports.
#/settings	API Keys	Seven independent encrypted agent key slots and provider testing.
#/guide	User Guide	Step-by-step operating instructions and troubleshooting.

2. OBJECTIVES AND SCOPE OF PROJECT

2.1 Primary Objectives:

• To design and develop an intuitive, multi-view SaaS web application ('Meridian') using clean, foundational technologies: HTML5, CSS3, JavaScript, and Python.

• To convert a single natural-language research query into a complete, board-ready market-intelligence report containing an executive summary, market trajectory, competitive share, SWOT, PESTLE, Porter's Five Forces, a 12-month forecast, and claim-level fact-checking — with real, clickable citations.

• To guarantee factual integrity through a Verified-Answer Policy: reports are generated only from a configured provider API and live retrieved sources; otherwise the system refuses with HTTP 409 instead of fabricating an answer.

• To support sixteen LLM providers through seven independent, Fernet-encrypted agent key slots, including automatic selection of the best currently-free OpenRouter model so the platform can run at zero API cost.

• To survive real-world free-tier failure modes — empty, truncated, or reasoning-only model completions and HTTP 429 rate limits — through typed completion errors, JSON repair, bounded retries, cooldown waits, and failover to another configured free provider.

• To export every report as PDF, PowerPoint, and Excel files, to answer follow-up questions through a grounded Chat Agent, and to translate reports into 14 supported languages including Hindi and Punjabi.

2.2 Scope of the Project:

The scope of Meridian covers automated market research for students, analysts, founders, and small strategy teams who cannot afford subscription research platforms. The system accepts any market, company comparison, region, and 1Y/3Y/5Y period; gathers live evidence from Tavily, NewsAPI, DuckDuckGo, the World Bank, and Reddit connectors; and returns an exportable, cited report. Meridian deliberately does not provide investment advice, does not execute trades, and clearly labels any chart series that could not be verified against a live source.

3. SYSTEM ARCHITECTURE & FEASIBILITY STUDY

Meridian adopts a layered Client–Server SaaS Architecture separating the presentation interface (HTML5/CSS3/JS single-page application and an alternate Streamlit console), the application tier (a Python FastAPI server exposing REST endpoints and Server-Sent Events), the multi-agent orchestration engine, and external data planes (live evidence connectors, sixteen LLM provider APIs, and a SQLite/PostgreSQL database).

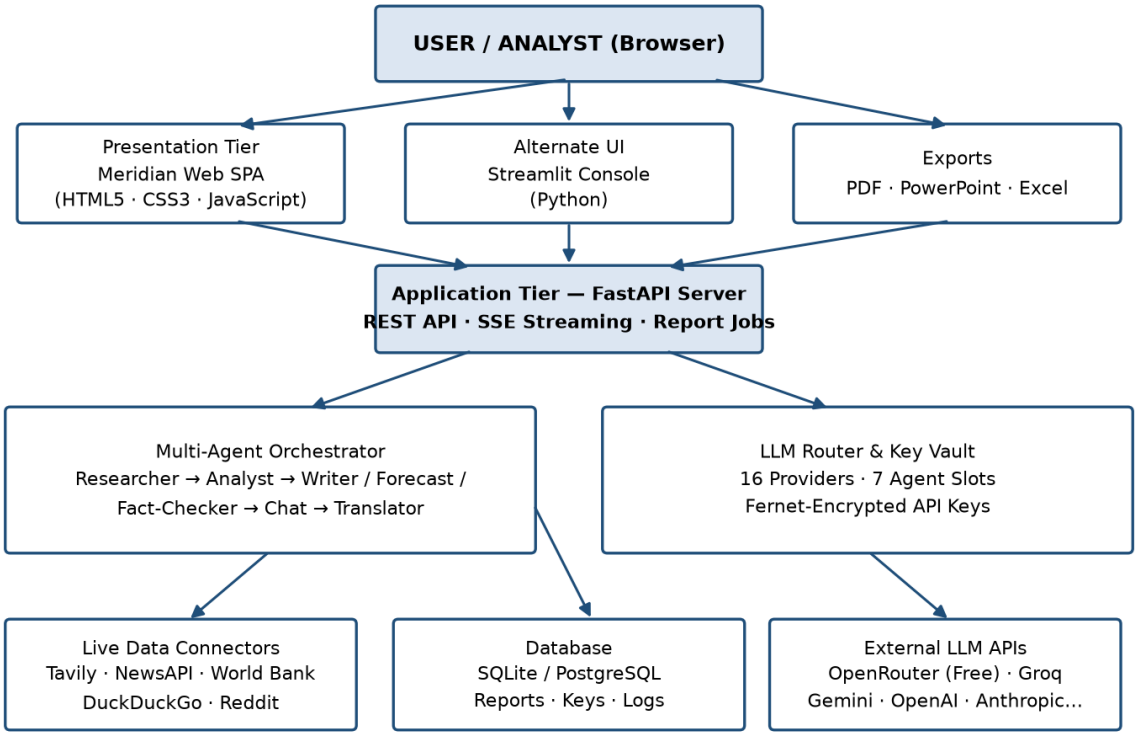


Figure 3.1: Meridian Layered Client–Server & Multi-Agent Architecture

Feasibility Analysis:

• **Technical Feasibility:** Built on standard cross-platform technologies (HTML5, CSS3, ES6 JavaScript, and Python 3 with FastAPI), requiring only a browser and an internet connection; no client installation or GPU is needed because inference runs on provider APIs.

• **Economic Feasibility:** The platform runs at zero recurring API cost on free-tier keys (OpenRouter :free models, Groq, Gemini); deployment on a free PaaS tier (Render) with SQLite storage keeps hosting cost at or near zero.

• **Operational Feasibility:** The interface follows familiar dashboard patterns — one query box, three selectors, and one Generate button — so non-technical users need no training, while the User Guide documents every flow.

• **Legal & Ethical Feasibility:** Only provider APIs used under their free tiers are integrated; reports cite real sources, label unverified figures, and carry a clear not-investment-advice disclaimer.

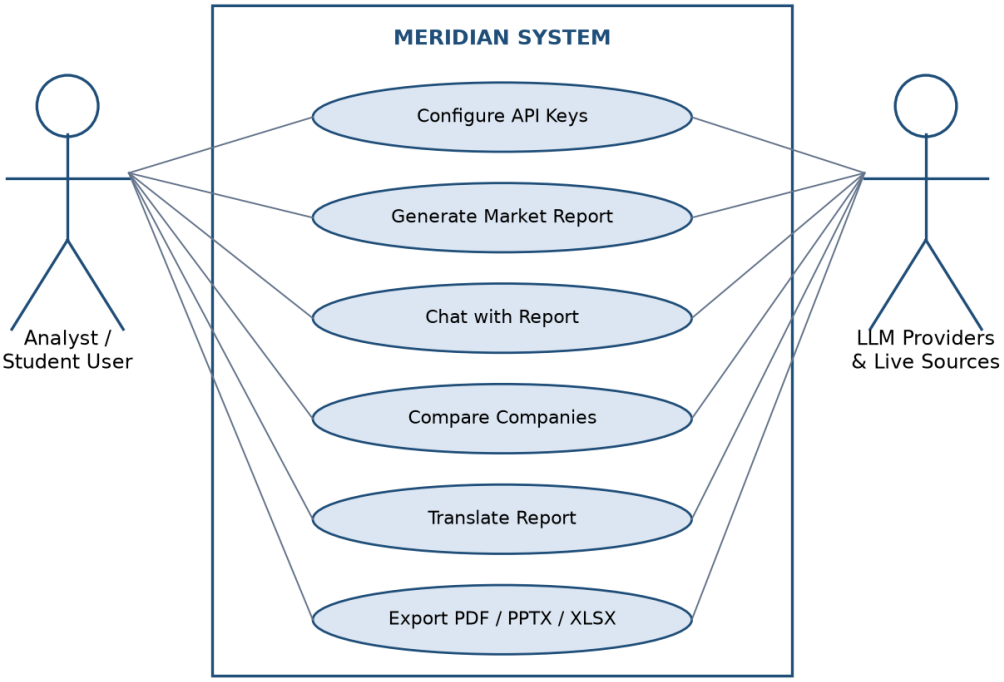


Figure 3.2: UML Use Case Diagram of Meridian

4. INTRODUCTION TO HTML5, CSS3, JAVASCRIPT & PYTHON

4.1 HTML5 (HyperText Markup Language)

HTML5 forms the structural backbone of Meridian's single-page application, organised into Dashboard, Report, History, Compare, Settings, and Guide views. Semantic elements such as <header>, <nav>, <main>, <section>, and <article> establish document hierarchy, while responsive meta configuration and accessible form labels make the console usable from a phone to an ultra-wide monitor without horizontal scrolling.

4.2 CSS3 (Cascading Style Sheets)

CSS3 powers Meridian's signature three-colour design system — Obsidian (#0B0906), Champagne Gold (#C6A15B), and Pearl (#F3ECDD) — using CSS custom properties, grid and flexbox layouts, sticky mission-control panels, and media queries that guarantee a universal fit on desktop and mobile with no overlapping elements and no bottom tab bar.

4.3 JavaScript (ES6+)

Vanilla JavaScript drives all client-side behaviour: hash-based routing, live Server-Sent Events consumption for agent telemetry, canvas/SVG chart rendering (line, donut, bar, heatmap), report filtering, comparison selection, encrypted-key settings management, and toast notifications — with no framework dependency, keeping the bundle small and auditable.

4.4 Python & FastAPI

Python 3 implements the entire backend: the FastAPI REST/SSE server, the seven-agent orchestration engine, the sixteen-provider LLM router with OpenRouter free-model auto-selection, the Fernet-encrypted key vault, live-data connectors, and the PDF (ReportLab), PowerPoint (python-pptx), and Excel (openpyxl) export generators. An alternate Streamlit interface is provided for a Python-native console.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements:

Component	Minimum Requirement	Recommended
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 (1.6 GHz+)	Intel Core i5 / Apple M1 or higher (2.4 GHz+)
Memory (RAM)	4 GB RAM	8 GB RAM or above
Storage (Disk)	2 GB free disk space	10 GB SSD free space
Display	1280 × 720 HD resolution	1920 × 1080 Full HD resolution
Network	Broadband / Wi-Fi internet connection	High-speed Wi-Fi / Ethernet for fast API calls

5.2 Software Requirements:

Software	Specification
Operating System	Windows 10/11, macOS, or any modern Linux distribution
Runtime	Python 3.11 or above
Backend Framework	FastAPI + Uvicorn (Streamlit for the alternate console)
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+) — any modern browser
Database	SQLite (default) / PostgreSQL (production)
Libraries	Requests, ReportLab, python-pptx, openpyxl, Cryptography (Fernet)
API Keys (free tier)	At least one: OpenRouter, Groq, or Google Gemini developer key
Development Tools	Visual Studio Code, Git, GitHub, Render (deployment)

6. CONCLUSION & EXPECTED OUTCOMES

Meridian demonstrates how foundational technologies — HTML5, CSS3, Vanilla JavaScript, and Python FastAPI — can be thoughtfully combined with modern free-tier Large Language Model APIs and a disciplined multi-agent design to deliver a consulting-grade research product. By refusing to fabricate answers, encrypting every credential, auto-selecting genuinely free models, and surviving the empty-completion, truncation, and rate-limit failures that free tiers produce in practice, Meridian is expected to empower students, founders, and analysts to obtain cited, exportable market intelligence in minutes at zero recurring cost.

7. REFERENCES

- MDN Web Docs — HTML5, CSS3 & Modern JavaScript Reference: https://developer.mozilla.org/
- Python 3 Official Documentation — Python Software Foundation: https://docs.python.org/3/
- FastAPI Official Documentation — Sebastián Ramírez: https://fastapi.tiangolo.com/
- OpenRouter API & Free Models Documentation: https://openrouter.ai/docs
- Groq Cloud OpenAI-Compatible API Reference: https://console.groq.com/docs
- Google AI Studio — Gemini API Official Documentation: https://ai.google.dev/gemini-api/docs
- W3Schools Online Web Tutorials (HTML, CSS, JS & Python): https://www.w3schools.com/
- ReportLab PDF Library Documentation: https://docs.reportlab.com/
- Streamlit Official Documentation: https://docs.streamlit.io/